

Ejercicio 01

cursos-controller

cursos-repository

cursos-service

ROL
Actua como desarrollador backend Node/Express.

CONTEXTO
arquitectura en capas, pg sin ORM, ES modules, y te voy a pasar cursos-repository.js, cursos-service.js y db-pg.js como referencia de estilo.

TAREA
Necesito que generes las 3 capas para materias

RESTRICCIONES
sin dependencias nuevas, mismo estilo (clases, delegar el acceso a datos en la clase DbPg con this.db.queryAll/queryOne/queryReturnId/queryRowCount, queries \$1), mantener los console.log.

ITERACIÓN
Pasame el primeramente el repository, lo reviso y luego te pido los siguientes.

+ Pregunta a Gemini

Flash

Tus conversaciones de Asociación ORT Argentina no se usan para mejorar nuestros modelos. Gemini es una IA y puede cometer errores. [Tu privacidad y Gemini](#)

RESPUESTAS:

1) REPOSITORY

¡Buenas! Me pongo el chip de backend. Entendido el enfoque: arquitectura en capas limpias, sin ORM, usando `db-pg.js` con parámetros posicionales (`$1`, `$2`), y manteniendo la consistencia estética (clases, funciones flecha asincrónicas y los `console.log`).

Como me pediste ir paso a paso, acá tenés la capa de acceso a datos para materias.

materias-repository.js

```
JavaScript

import Db from './db-pg.js';

export default class MateriasRepository {
  constructor() {
    console.log('Estoy en: MateriasRepository.constructor()');
    this.db = new Db();
  }

  getAllAsync = async () => {
    console.log('MateriasRepository.getAllAsync()');
    const sql = 'SELECT * FROM materias';
    return await this.db.queryAll(sql);
  }

  getByIdAsync = async (id) => {
    console.log('MateriasRepository.getByIdAsync(${id})');
    const sql = 'SELECT * FROM materias WHERE id=$1';
    return await this.db.queryOne(sql, [id]);
  }

  createAsync = async (entity) => {
    console.log('MateriasRepository.createAsync(${JSON.stringify(entity)})');
    const sql = 'INSERT INTO materias (nombre) VALUES ($1) RETURNING id';
    const values = [entity?.nombre ?? ''];
    return await this.db.queryReturnId(sql, values);
  }

  updateAsync = async (entity) => {
    console.log('MateriasRepository.updateAsync(${JSON.stringify(entity)})');
    const sql = 'UPDATE materias SET nombre = $2 WHERE id = $1';
    const values = [
      entity.id,
      entity?.nombre ?? ''
    ];
    return await this.db.queryRowCount(sql, values);
  }

  deleteByIdAsync = async (id) => {
    console.log('MateriasRepository.deleteByIdAsync(${id})');
    const sql = 'DELETE FROM materias WHERE id=$1';
    return await this.db.queryRowCount(sql, [id]);
  }
}
```

Nota: Asumí que la tabla se llama `materias` y que, al igual que los cursos, por ahora cuenta con un campo `nombre`. Si llega a tener más campos (como una `FK` a cursos, carga horaria, etc.), avisame y lo adaptamos al toque.

Revisalo tranquilo. Cuando te cierre, pasamos al Service.

2) SERVICE

Pasamos al service

¡Excelente! Seguimos con la capa de negocio. Mantenemos el mismo patrón: inyección del repositorio en el constructor, funciones flecha y la delegación directa de la lógica con sus respectivos logs.

materias-service.js

JavaScript

```
import MateriasRepository from '../repositories/materias-repository.js';

export default class MateriasService {
  constructor() {
    console.log('Estoy en: MateriasService.constructor()');
    this.MateriasRepository = new MateriasRepository();
  }

  getAllAsync = async () => {
    console.log('MateriasService.getAllAsync()');
    const returnArray = await this.MateriasRepository.getAllAsync();
    return returnArray;
  }

  getByIdAsync = async (id) => {
    console.log('MateriasService.getByIdAsync(${id})');
    const returnEntity = await this.MateriasRepository.getByIdAsync(id);
    return returnEntity;
  }

  createAsync = async (entity) => {
    console.log('MateriasService.createAsync(${JSON.stringify(entity)})');
    const rowsAffected = await this.MateriasRepository.createAsync(entity);
    return rowsAffected;
  }

  updateAsync = async (entity) => {
    console.log('MateriasService.updateAsync(${JSON.stringify(entity)})');
    const rowsAffected = await this.MateriasRepository.updateAsync(entity);
    return rowsAffected;
  }

  deleteByIdAsync = async (id) => {
    console.log('MateriasService.deleteByIdAsync(${id})');
    const rowsAffected = await this.MateriasRepository.deleteByIdAsync(id);
    return rowsAffected;
  }
}
```

3) CONTROLLER

¡Venga ese controller! Cerramos el circuito de materias manteniendo el ruteo de Express, el manejo de estados con `http-status-codes`, las validaciones de IDs en el `PUT` y la captura de errores en los bloques `catch`.

materias-controller.js

JavaScript

```
import { Router } from 'express';
import { StatusCodes } from 'http-status-codes';
import MateriasService from '../services/materias-service.js'

const router = Router();
const currentService = new MateriasService();

router.get('', async (req, res) => {
  try {
    console.log('MateriasController.get');
    const returnArray = await currentService.getAllAsync();
    if (returnArray !== null){
      res.status(StatusCodes.OK).json(returnArray);
    } else {
      res.status(StatusCodes.INTERNAL_SERVER_ERROR).send('Error interno.');
```

```
    }
  } catch (error) {
    console.log(error);
    res.status(StatusCodes.INTERNAL_SERVER_ERROR).send(`Error: ${error.message}`);
  }
});

router.get('/:id', async (req, res) => {
  try {
    let id = req.params.id;
    const returnEntity = await currentService.getByIdAsync(id);
    if (returnEntity !== null){
      res.status(StatusCodes.OK).json(returnEntity);
    } else {
      res.status(StatusCodes.NOT_FOUND).send('No se encontro la entidad (id:${id})');
    }
  } catch (error) {
    console.log(error);
    res.status(StatusCodes.INTERNAL_SERVER_ERROR).send(`Error: ${error.message}`);
  }
});
```

```

router.post('', async (req, res) => {
  try {
    let entity = req.body;
    const newId = await currentService.createAsync(entity);
    if (newId > 0 ){
      res.status(StatusCodes.CREATED).json(newId);
    } else {
      res.status(StatusCodes.BAD_REQUEST).json(null);
    }
  } catch (error) {
    console.log(error);
    res.status(StatusCodes.BAD_REQUEST).send(`Error: ${error.message}`);
  }
});

router.put('/:id', async (req, res) => {
  try {
    let id = parseInt(req.params.id);
    let entity = req.body;

    if (entity.id && parseInt(entity.id) !== id) {
      return res.status(StatusCodes.BAD_REQUEST).send(`El id de la URL (${id})`);
    }

    entity.id = id;
    const rowsAffected = await currentService.updateAsync(entity);
    if (rowsAffected !== 0){
      res.status(StatusCodes.OK).json(rowsAffected);
    } else {
      res.status(StatusCodes.NOT_FOUND).send(`No se encontro la entidad (id:${id})`);
    }
  } catch (error) {
    console.log(error);
    res.status(StatusCodes.BAD_REQUEST).send(`Error: ${error.message}`);
  }
});

router.delete('/:id', async (req, res) => {
  try {
    let id = req.params.id;
    const rowCount = await currentService.deleteByIdAsync(id);
    if (rowCount !== 0){
      res.status(StatusCodes.OK).json(null);
    } else {
      res.status(StatusCodes.NOT_FOUND).send(`No se encontro la entidad (id:${id})`);
    }
  } catch (error) {
    console.log(error);
    res.status(StatusCodes.INTERNAL_SERVER_ERROR).send(`Error: ${error.message}`);
  }
});

export default router;

```